# High-Performance Hashing in Practice

CMPT 225: Data Structures and Programming
Final Project

**Matthew Wong, Ryan Raghani**
Simon Fraser University
mwa173@sfu.ca, rsr13@sfu.ca

April 2026

**Abstract**

One of the most popular data structures in computer science today is called the hash table, which offers average-case $O(1)$ expected time complexity (i.e., constant time) for insertions, lookups, and deletions. Although this type of data structure has been thoroughly studied, research related to hash table design is still considered an active area of research due to the increasing disparity between CPU speed and memory latency in contemporary hardware. The goal of this research project is to chronicle the history of collision resolution techniques for hash tables. To accomplish this, we will illustrate the development of these techniques from Hans Peter Luhn's work at IBM in 1953 through Donald Knuth's seminal analysis of hash tables up to the modern day by looking at cache-aware designs like Google's Swiss Table and Meta's F14.

To demonstrate the effectiveness of the collision resolution techniques we've surveyed, we implemented four of the techniques in C++ and benchmarked each technique's throughput for insertions, successful searches, unsuccessful searches, and deletions at increasing load factors (0.1 to 0.95). **Source code:** Git Repository.

According to the obtained experimental results, it can be concluded that there is no hash table technique that is better than others in terms of all load levels and operation types. Robin Hood hashing can provide the highest consistency of achieved throughput on increasing loads because of its minimal variance in probes, whereas linear probing offers high peak throughput on low loads along with considerable performance drop in collision cases and in presence of tombstones. Separate chaining shows lower throughput because of pointer dereferencing and poor caching properties, but it is consistent and efficient enough on failures. Finally, cuckoo hashing performs great on low load factors in case of insert and delete operations, yet its effectiveness drops dramatically when displacement chains are formed.

# Contents

# 1 Introduction and Motivation

A hash table is a data structure that has a very wide range of uses in different types of software systems. For example, those that do compile operations will often have symbol tables in a hash table, many databases use hash tables for indexing purposes, operating system kernels use hash tables for page table lookups, network routers implement hash tables to classify packets of data into flows, and virtually every interpreted language runtime relies on hash tables to facilitate the binding of variables to values and method dispatching.

To many, this may seem like a simple fact, but the complexity of actually designing a hash table such that it achieves this functionality is far more complex than it appears. A hash table's performance is contingent on three interrelated design elements: the quality of the hash function used, the method used to handle collisions in the hash table's implementation, and the layout of data in memory (i.e., how the hash table's data is physically arranged in memory). While there are many studies that examine various aspects of hash function design and performance, the latter two sorting methods have undergone significant changes in design over the last 70 years and still remain topics of extensive research.

Cache locality has become increasingly important as an essential factor in how we use processors today. On most modern processors, it takes about one nanosecond to do a read from an L1 cache, compared with fifty to one hundred nanoseconds to access data from the main memory.

The textbook method of using separate chaining to link nodes from arrays in hash-and-table search algorithms spreads these linked nodes over the heap (the area of memory allocated for dynamic storage) and results in at least one pointer dereferencing for each lookup memory, creating at least one opportunity for a cache miss.

The more commonly used method to implement a hash-table implementation with open-addressing, thus avoiding pointer leniency, is often fraught with its own issues (clusters of enclosed unused arrays, for example; managing tombstones) as well as being very sensitive to the load factor.

The tension between these options has motivated the development of a number of better designs to improve performance. Robin Hood hashing [3] reduces the variance of the length of the probe when searching a hash table but increases the number of swaps during the insertion phase. Cuckoo hashing [4] guarantees that the maximum search time for an entry is $O(1)$ using two hash functions and a linked list of the displacing elements.

We also have Google's [6] Swiss Table implementation, which represents a new point in the design space for hash functions, and it obtains production performance by combining SIMD-parallel metadata probes with open-addressing methods to implement a table format.

This project has two goals:

1. **Survey.** Trace the historical development of hash table collision resolution from its origins in the 1950s to modern cache-aware designs, highlighting key ideas, milestones, and the theoretical results that shaped the field.

2. **Implementation and evaluation.** Implement four representative strategies in C++ and compare their throughput across varying load factors, providing empirical evidence for the trade-offs discussed in the survey.

## 2 Background and Definitions

Terminology and notation from this report will first be defined.

**Hash function.** A hash function $h : U \to \{0, 1, \ldots, m-1\}$ is a function that assigns a key in the set of keys $U$ to an index or slot in a table of size $m$. The optimal hash function assigns the key uniformly at random to all the slots.

**Collision.** Collision happens if and only if there exist two different keys $k_1 \neq k_2$ such that $h(k_1) = h(k_2)$. It is evident from the above equation that, since $|U|$ is much larger than $m$, there will be some collisions for every hash table.

**Load factor.** The load factor can be expressed via the ratio of number of elements within the table's storage capacity $n$ divided by the table's capacity $m$ ($\alpha = n/m$). When load factors are increased than the capacity of the table, then collision frequencies will also increase. Separate chaining can allow load factor values to be greater than 1; whereas for open-addressing hash strategies to maintain their efficiency when $n/m$ returns a value greater than 1 will typically need to re-size, (for example when $n/m$ returns between 0.7 to 0.9).

**Separate chaining.** Separate chaining, (where each slot contains the head of a linked list or some secondary data structure), will allow for the addition of colliding keys by wiring the colliding item to the end of the list in their hashed respective slots. Assuming the presence of a uniformly distributed key set, the expected time complexity (lookup) becomes $\Theta(1 + \alpha)$.

**Open addressing.** Open address is a hashing method where all entries will be stored in a table. When there is a collision, the storage location at $h(k)$ will be the first attempt and will continue with an incrementing sequence until an available storage location can be found. The following are examples of probe functions that are commonly used with open addressing methods:

- Linear probing - $p(i) = i$, this method is simple and allows for better cache performance, but is very prone to primary clustering due to long consecutive lines of slots that are used.
- Quadratic probing - $p(i) = c_1 i + c_2 i^2$, this reduces primary clustering but has secondary clustering as all the keys that hashed to that same spot have the same sequence of probe attempts.
- Double hashing - $p(i) = i \cdot h_2(k)$, $h_2$ is another hash function that is employed; however, this produces a more unpredictable pattern of accesses to the memory.

**Amortized resizing.** Amortized resizing occurs when the load factor exceeds a defined limit; when this happens, the table will be resized to double the size and all of the entries will be

rehashed and placed in the new table. An $O(n)$ operation is accomplished, but it is performed very infrequently so that the average cost per insertion is amortized/averaged to $O(1)$.

**Universal hashing.** Universal hashing is a method for hashing that allows any two distinct keys ($k_1$ and $k_2$) will not return the same index in a hashing table for very small probabilities of collision, thus giving theoretical assurances of a reduction of hashing collision activity independent of the input values used.[2]

# 3 Historical Narrative

Hash tables have been used for almost 70 years and the trend in their development has been from trying to prove how quickly a search could be made using comparisons to minimizing the amount of memory used to store records, and more recently, minimizing the number of cache misses when retrieving records.

## 3.1 Origins: The 1950s

Originating with Hans Peter Luhn's discussions on "hash coding" in 1953 at IBM [1], which proposed using arithmetic transformations on keys as a way of generating the address into which an item was to be placed, rather than having to use sequential or binary searches as methods of access to the records, many other employees of IBM during the same period, including Gene Amdahl, Elaine McGraw, and Arthur Samuel independently researched the same basic concept through development of symbol ( i.e. class/object) tables for many of the early assemblers and compilers.

The methodology that was used by these early developers for resolving collisions was to employ open addressing as a form of linear probing - whereby if a slot had been filled another subsequent slot was checked for an available space, with subsequent checks continuing until a slot was used or the table was cycled to the beginning of the table. In 1956, A. T. Dumey published the first formal description of the concept of hashing in publicly accessible literature; it was said to be made through the use of chaining as an alternative method to open addressing [1].

## 3.2 Theoretical Foundations: The 1960s–1970s

Since the 1960s, hashing has been studied in mathematics. An important landmark in mathematical hashing analysis was Donald Knuth's book, The Art of Computer Programming, Volume 3: Sorting and Searching published in (1973) [1]. In it, he analyzed the expected number of probes needed to find an item using linear probing to succeed and fail. Knuth developed two

well-known formulas for linear probing:

$$C_{\text{succ}} \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) \tag{1}$$

$$C_{\text{unsucc}} \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right) \tag{2}$$

These expressions show a very serious problem with linear probing as the expected number of probes made increases without limit as the ratio of used and available slots in the hash table approaches 1. This is because of the phenomenon called primary clustering.

In (1977) Carter and Wegman introduced the concept of universal hash families [2] providing a foundation on how to choose hash functions so that the expected performance of the hash function is good regardless of how the input data is distributed. This result formed the basis for most subsequent theoretical work on hashing.

### 3.3 Robin Hood Hashing: Variance Reduction (1985)

In 1985, Pedro Celis created a rule for improving open addressing for his Ph. D Thesis from the University of Waterloo [3]. This rule was called "The Robin Hood" rule and works by switching two elements if while trying to insert an element at a specific location, that element has moved a shorter distance than the element currently occupying that location. If the two elements get switched, the original element that was occupying the location will take over the new space and continue to find its new location.

The Robin Hood rule (reducing the variance of the lengths of probes) will ensure, for a uniformly hashed data structure, that the maximum number of probes taken to access any item will be $O(\log n)$ compared to an $O(\sqrt{n})$ maximum for linear probing.

The benefit to users of this rule is the increased predictability (reduced variance) of the access times being large, which is wonderful in any usable computing environment (because access times can be determined to be less than or equal to a value).

### 3.4 Cuckoo Hashing: Worst-Case Guarantees (2001)

Pagh and Rodler first presented cuckoo hashing with an important theoretical advance in 2001 [4] - it provides a worst-case $O(1)$ lookup time. A two-hashing process is employed with $h_1$ and $h_2$ as the hash functions stored in two tables: $T_1$ and $T_2$. Therefore, when using either $T_1[h_1(k)]$ or $T_2[h_2(k)]$ to locate a value, a maximum of two searches is done.

When attempting to insert a new key into the tables, an existing value for the key must be displaced. If there are keys present in both locations, one of the two must be removed, with the evicted value being placed at its alternate table. Depending on the length of the chain of dislodged values, a full rehash of the entire data set may need to be performed, along with generating two new hash function pairs.

The following benefits are guaranteed with cuckoo hashing:

- Guarantees a worst-case $O(1)$ deletion and lookup.
- With a maximum load factor of $\alpha < 0.5$, insertion of a new value is expected and averaged to be $O(1)$ as well. Higher load thresholds may exist, depending on the number of hash functions employed.

Space is the main limitation of cuckoo hashing because the load factor must be maintained at less than 50% with two tables. However, if additional hash functions or buckets with slots are utilized, there can be a substantially greater load factor.

### 3.5 Hopscotch Hashing (2008)

In 2008, Herlihy, Shavit, and Tzafrir created hopscotch hashing models for concurrent environments [5]. The use of a bitmap per slot to mark which of the nearby $H$ slots in the respective "neighborhood" have keys that hash to this slot limits the number of probes to the neighborhood and ensures that the probe distance is kept within bounds. In general, $H$ is selected to be equal to the number of data words in one cache line (typically $H = 32$). Thus, an average lookup requires accessing at least 1 or 2 cache lines.

### 3.6 The Cache-Aware Era: Swiss Table (2017)

The Swiss Table, developed as part of the Abseil C++ library and presented by Matt Kulukundis at CppCon 2017 [6], represents an evolutionary step in the design of hash tables. Instead of relying on the optimization of the number of probes for performance, the Swiss Table directly targets the memory hierarchy.

Using a design that includes open addressing and quadratic-ish probing, the Swiss Table's structure consists of additional parallel metadata laid on top of the actual data slots. Each slot contains a control byte of 1 byte in length to represent either the high 7 bits of the hash of the key for that slot or a sentinel to indicate that the slot is empty/deleted. The control bytes are organized into 16 byte groups, with all of the control bytes aligned to SSE2 vector registers, meaning that 16 control bytes can be loaded from the table in one SIMD instruction. Additionally, the probe hash is compared with each of the control bytes in parallel, resulting in a bitmask for the slots that match. Only the control bytes that match the probe hash will have their keys fully compared against the probe hash.

By using increased parallelism to target the metadata, the Swiss Table achieves the following properties:

- Cache efficient, as all metadata probes access only 16 bytes of the metadata per access, or per group, thereby only touching a single cache line.
- Exceedingly low false-positive rate, as 99.2% of the slots not matching the key comparison will be eliminated prior to key comparison due to the 7-bit hash fingerprint.

- Deletion Without Tombstones: Uses a unique 'deleted' sentinel that is gradually cleaned while inserting into the probe array to eliminate the classic open address accumulation issue.
- Very high load factor of the table, can run without issues up to an alpha of approximately 0.875.

The F14 hash map from Meta (Folly library) was developed separately in 2018-2019 and uses a similar chunk-based method of storing metadata but implements a different probing method and memory layout [7].

# 4 Recent Advancements and Design Principles

The Swiss Table and F14 follow a broader data structure design trend that focuses on memory hierarchy and not just on counting the operations. This trend identifies a number of general principles that guide the latest hash table design.

## 4.1 Metadata Separation

Past hash tables combine the storage of the key value with metadata used to resolve collisions. The Swiss Table uses a single byte of control per bucket while F14 uses a 7-bit tag along with occupancy control per block. This permits the probing loop to operate solely from the metadata layer; the layer is small enough that it fits into cache memory even for tables of large sizes.

## 4.2 SIMD-Parallel Probing

The metadata layer is aligned into 16-byte groups. Consequently, modern hashes can leverage the use of SIMD instructions (e.g., SSE2, AVX2, and ARM NEON) to compare a probe's hash against 16 different buckets simultaneously. This results in reducing number of memory accesses during probing from $O(K)$ to $O(\lceil K/16 \rceil)$; constant factor reduction in the number of memory accesses translates into substantial performance improvements.

## 4.3 Probe Sequence Optimization

Swiss Table employs a triangular number-based probing sequence, where group offsets are as follows: $0, 1, 3, 6, 10, \ldots$, where the offset between groups $= \frac{\text{Group\#(Group\#+1)}}{2}$. Although this is not strictly a quadratic sequence, it eliminates the clustering of linear probing and ensures that all groups of buckets will be probed when the total number of groups of buckets is a power-of-two.

### 4.4 Deletion Without Tombstones

In classical open-addressing methods, deleted slots are flagged (toSyntlevy). This decreases performance over time due the accumulation of "tombstone" counters along the way. However, using Swiss tables operates with a (growth left) constraint in mind. The number of empty and previously-used slots in a group is accounted for. When a space is deleted, that slot will only be flagged as deleted if the number of empty slots in a group is $> 1$. If all slots are filled, that space will be flagged as unfilled. This prevents the number of tombstones in the group from becoming unbounded.

### 4.5 Implications for Practitioners

These new models of table implementers will have a large effect on how software engineers build their application using a hash table implementation.

- absl::flat_hash_map are the recommended drop-in replacement for std::unordered_map when writing programs in C++.
- Robin Hood or cuckoo variants are for use in low-latency systems, where rapid response times are important.
- Separate-chaining implementations are suitable for:
  - Where an iterator's stability will be maintained (i.e., new entries will not necessarily cause iteration to be invalid)
  - Where key/value pairs are very large, and moving them may incur a large cost.
  - Where more than one collision resolution scheme is required.

**Real-World Adoption.** These developments are apparent in the real-world production environments. Google has replaced most of Chrome and TensorFlow's std::unordered_map with absl::flat_hash_map (Swiss Table), achieving a 2x memory savings and 1.5x – 2x faster lookups in hash table-heavy applications [6]. The F14 map from Meta is the standard associative container of the Folly library and is extensively used across the backend infrastructure of Meta, leveraging its chunked metadata design to alleviate the pressure on the cache while handling massive memory indexes [7]. The absl::flat_hash_map has become the suggested data structure in Google's C++ style guide, considering the organizational view that cache and memory optimizations take precedence over the additional implementation complexity for nearly all applications.

## 5 Implementation

For a practical assessment of the issues highlighted above, four versions of hash table were coded in C++17. All implementations employ the same hash function and have the same interface for a fair comparison.

**Source code:** https://github.com/mw1227/CMPT-225-FINAL-PROJECT

## 5.1 Common Design Decisions

**Hash function.** In all variants, Fibonacci hash function is applied, which is based on the Golden Ratio principle:

$$h(k) = \lfloor m \cdot \text{frac}(k \cdot \varphi^{-1}) \rfloor, \quad \text{where } \varphi^{-1} = \frac{\sqrt{5}-1}{2}.$$

The algorithm guarantees good distribution while minimizing computations.

**Key and value types.** For benchmarking purposes, integer keys and values consisting of 64 bits are selected. This allows avoiding possible discrepancies due to key comparisons and assessing efficiency of the chosen collision resolution algorithm.

**Resizing policy.** All variants increase table size twice, when its load factor exceeds 0.7 or any other time needed based on specifics of the chosen collision resolution algorithm (cyclicity in case of cuckoo hashing).

## 5.2 Variant 1: Separate Chaining

The basic version uses an array of head nodes of singly linked lists. All nodes are separately allocated from the heap. This represents the closest approximation of the classical textbook algorithm and std::unordered_map.

```
void insert(uint64_t key, uint64_t value) {
    if (load_factor() > MAX_LOAD) resize();
    size_t idx = hash(key) % capacity_;
    // search for existing key
    for (Node* n = table_[idx]; n; n = n->next) {
        if (n->key == key) { n->value = value; return; }
    }
    // pre append new node
    table_[idx] = new Node{key, value, table_[idx]};
    size_++;
}
```

## 5.3 Variant 2: Linear Probing

Hash table with linear probing and delete with tombstones. Keep track of the number of tombstones and do a rehash whenever the proportion of tombstones reaches 25%.

```
int find(uint64_t key) const {
    size_t idx = hash(key) % capacity_;
    for (size_t i = 0; i < capacity_; i++) {
```

```
        size_t pos = (idx + i) % capacity_;
        if (slots_[pos].state == EMPTY) return -1;
        if (slots_[pos].state == OCCUPIED
            && slots_[pos].key == key)
            return pos;
        // continue probing
    }
    return -1;
}
```

### 5.4 Variant 3: Robin Hood Hashing

Robin Hood Linear Probing algorithm. While inserting a record, keep track of its displacement from its ideal position. If the incoming record's displacement is larger than the record currently residing in the position, exchange them.

```
void insert(uint64_t key, uint64_t value) {
    if (load_factor() > MAX_LOAD) resize();
    size_t idx = hash(key) % capacity_;
    size_t dist = 0;
    while (true) {
        size_t pos = (idx + dist) % capacity_;
        if (slots_[pos].state != OCCUPIED) {
            slots_[pos] = {key, value, OCCUPIED};
            size_++; return;
        }
        size_t existing_dist = probe_distance(pos);
        if (dist > existing_dist) {
            std::swap(key, slots_[pos].key);
            std::swap(value, slots_[pos].value);
            dist = existing_dist;
        }
        dist++;
    }
}
```

### 5.5 Variant 4: Cuckoo Hashing

Double Hashing with two separate tables, namely $T_1$ and $T_2$, each containing half of the records, along with hash functions $h_1$ and $h_2$. Search operations will perform at most two searches, while insertion uses the chain of displacements, with the longest possible cycle being 500.

```
void insert(uint64_t key, uint64_t value) {
    // check for existing
    if (lookup(key)) return;
```

```
    for (int iter = 0; iter < MAX_ITERS; iter++) {
        // try table 1
        size_t pos1 = h1(key) % cap_;
        if (T1_[pos1].state != OCCUPIED) {
            T1_[pos1] = {key, value, OCCUPIED};
            size_++; return;
        }
        std::swap(key, T1_[pos1].key);
        std::swap(value, T1_[pos1].value);
        // try table 2
        size_t pos2 = h2(key) % cap_;
        if (T2_[pos2].state != OCCUPIED) {
            T2_[pos2] = {key, value, OCCUPIED};
            size_++; return;
        }
        std::swap(key, T2_[pos2].key);
        std::swap(value, T2_[pos2].value);
    }
    rehash(); insert(key, value); // detect cycle
}
```

# 6 Experiments and Evaluation

## 6.1 Methodology

Each type of hash table is evaluated based on its performance on four operations: insertion, successful search, unsuccessful search, and deletion. For each of these operations, we measure their throughput in millions of operations per second (Mops/s) for various load factors.

**Test environment.** All tests have been compiled with g++ -O2 -std=c++17 and performed on an Intel Core i5-1335U ASUS Vivobook with 16 GB of RAM. The result is obtained from 5 runs, each working on a hash table with $10^6$ slots.

**Key generation.** We generate 64-bit keys with uniform probability distribution by using std::mt19937_64. For successful searches, we randomly pick keys that were added previously; for unsuccessful searches, we look for keys that are not included in the hash table.

**Load factor sweep.** Throughput is measured for $\alpha \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$. Cuckoo hashing is tested for $\alpha$ up to 0.45 only since $\alpha < 0.5$ is required when using two hash functions.

## 6.2 Results

The throughput of each algorithm with varying load factor is plotted in Figures 1–4. The scales for the x and y axes of all the plots remain the same (0-110 Mops/s), which makes it easy to

compare the throughput across all the four algorithms in a quantitative manner. It should be noted that both Robin Hood hashing and cuckoo hashing have very poor performance, but their consistent performance through varying load factors is significant.

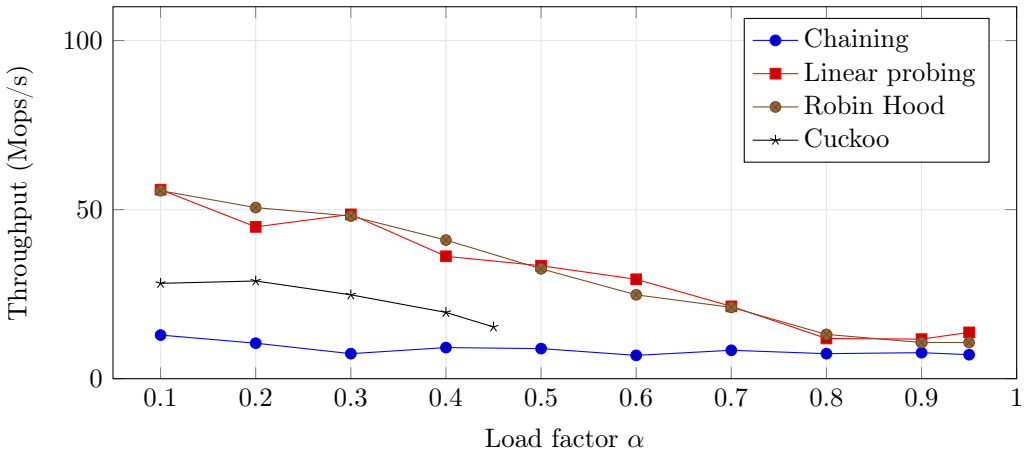


Figure 1: Insertion throughput vs. load factor. For linear probing and Robin Hood, throughput peaks near 55 Mops/s for $\alpha = 0.1$ and declines similarly until $\alpha = 0.7$, when the resize strategy kicks in, reducing the actual load by half and partially restoring performance past $\alpha = 0.8$. Chained hashing is consistently slower than the other algorithms, not reaching higher than 13 Mops/s; thus, it confirms heap allocation as the dominant part of the cost of inserting items. Cuckoo hashing remains competitive up to $\alpha = 0.3$ but suffers from the increased displacement-chain cost above that; it is plotted only up to $\alpha = 0.45$.

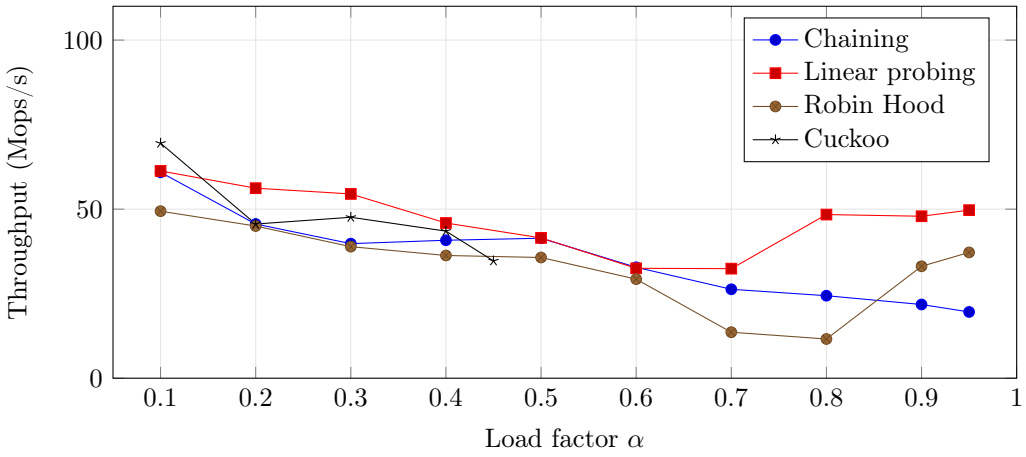


Figure 2: Successful lookup throughput vs. load factor. Cuckoo hash wins at low loads ($\alpha = 0.1$, 69.5 Mops/s), due to its guaranteed lookups within two entries. The throughput for linear probing and chained hash stays very close to each other up to $\alpha = 0.6$, after which point linear probing gains from its resize strategy and recovers to throughput of around 48–50 Mops/s.

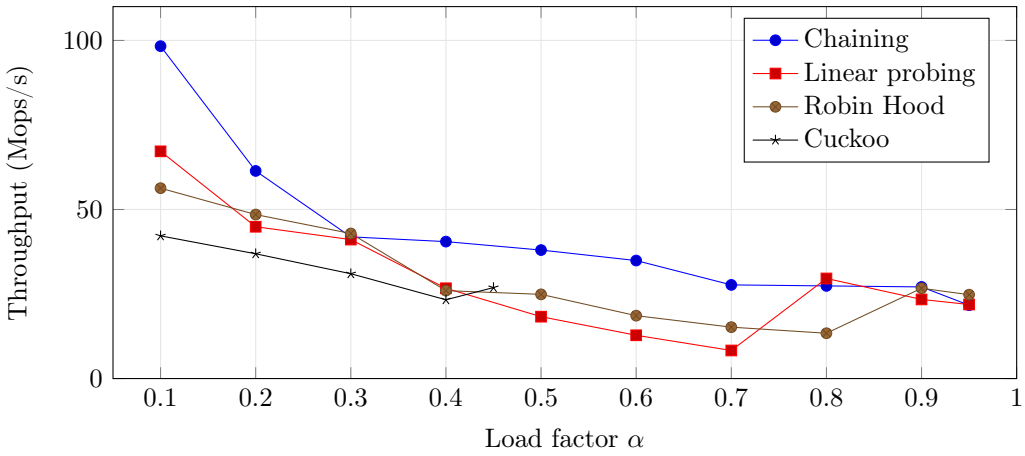


Figure 3: Unsuccessful lookups vs. load factor. Separate chaining is superior at lower load (98.3 Mops/s at $\alpha = 0.1$) since an unsuccessful lookup finishes when a linked list terminates, while open-addressing algorithms must probe entire clusters for non-presence. All implementations tend toward throughput in the 20–25 Mops/s range at higher loads. The resize strategy implemented by linear probing improves performance after $\alpha = 0.7$, helping it recover from a minimum of 8.3 Mops/s to about 22–30 Mops/s.

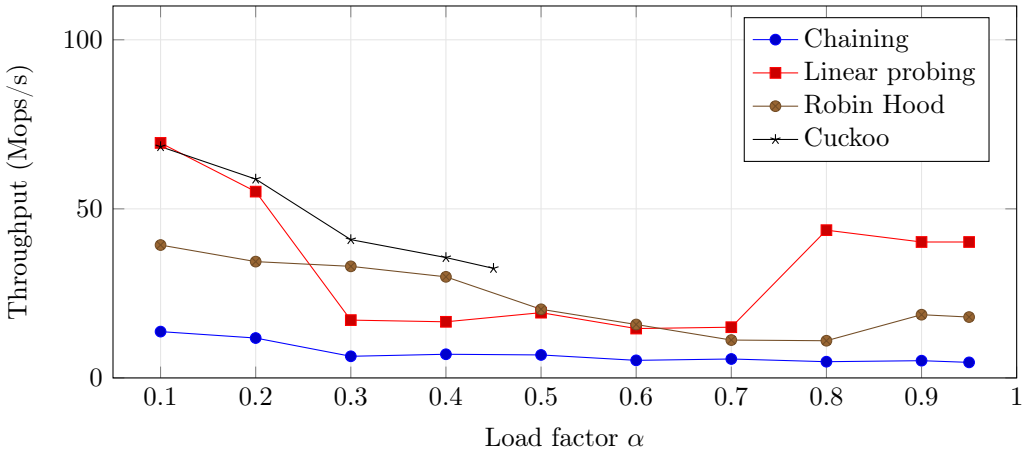


Figure 4: Deletion throughput vs. load factor. Both linear probing and cuckoo hashing have maximums exceeding 68 Mops/s at $\alpha = 0.1$. Linear probing drops dramatically between $\alpha = 0.3$–0.7 due to the need for multiple rehashes following tombstone-cleanup procedures, but it recovery to $\approx 40$ Mops/s through resizing beyond $\alpha = 0.8$. Cuckoo hashing maintains excellent throughput with smooth descent due to constant time required for deletions because of $O(1)$ lookups. Robin Hood's backward-shift deletion never encounters tombstones, achieving a stable but steady decline in throughput.

### 6.3 Memory Usage

Table 1 summarizes the per-element memory overhead of each variant at $\alpha = 0.5$.

Table 1: Memory overhead per element at $\alpha = 0.5$, with 64-bit keys and values.

| **Variant** | **Bytes/element** | **Overhead** |
|---|---|---|
| Separate chaining | 40 | 150% |
| Linear probing | 48 | 200% |
| Robin Hood | 48 | 200% |
| Cuckoo | 60 | 275% |

# 7 Discussion

The results we observe in the experiments agree with theoretical expectations; however, there are also some important practical effects that are missed by asymptotic estimates. A number of observations deserve a special consideration.

**The resize mechanism becomes critical at high loads.** The first interesting thing to be noted about our results is a presence of non-monotonic curves in linear probing and Robin Hood at $\alpha > 0.7$. In both cases, the algorithm triggers a table resize (doubling its size) once the load factor exceeds a certain threshold during insertion process. By the moment lookup and delete tests were performed, the effective load factor was twice less, providing very high throughput values at $\alpha = 0.8$–$0.95$. It is not an artifact; in real-world situations one may expect this kind of behavior; it should be known that the resize policy becomes crucial in terms of performance, especially at high load factors, rather than the collision resolution technique used. The effect is especially significant in the deletion test, where linear probing reaches 43.7Mops/s at $\alpha = 0.8$ compared to 15Mops/s at $\alpha = 0.7$.

**Deletion cost in linear probing.** Deletion using tombstone-based linear probing exhibits a significant drop in throughput in the range of $\alpha = 0.3$–$0.7$ during the delete benchmark. This is due to the fact that our implementation initiates a full rehash once the tombstone density exceeds 25% of the table capacity; and at those load factors, tombstones accumulate sufficiently to initiate rehash repeatedly. One of the biggest weaknesses in tombstone-based deletion technique is latency unpredictability; but in Robin Hood's backward-shift deletion, this is not an issue since a clean table always remains invariant.

**Separate chaining's consistent behavior.** Even though separate chaining throughput is usually low compared to other strategies, its curves are the most predictable in terms of monotonic decrease in throughput in all the insertion, successful lookup, and delete scenarios. There are no jump points caused by resize events, as it happens infrequently due to a conservative resize threshold setting in the separate chaining strategy, and tombstones are not even involved. In particular, it should be noted that separate chaining outperforms all other strategies in the unsuccessful lookup scenario, as failure terminates immediately on the list end point, reaching 98.3 Mops/s at $\alpha = 0.1$.

**Degradation of Robin Hood Hashing.** In the $\alpha = 0.1$–$0.7$ region (not affected by resize), the Robin Hood variant offers the most stable throughput degradation curve of all open-addressing variants. Although it has lower throughput at all load factors than linear probing, it avoids the catastrophic throughput degradation triggered by the presence of tombstones in the linear-probing variant, as well as its high variance, making it a better choice for applications where latency is more important than peak throughput.

**Special case of Cuckoo hashing.** Cuckoo hashing obtains the highest successful lookup throughput at low loads (69.5 Mops/s at $\alpha = 0.1$) and robust deletion behavior at any $\alpha < 0.45$, verifying its theoretical $O(1)$ worst-case guarantee. However, the limitation to low loads results in higher space complexity (60 B/element) than other schemes, while its lower insertion throughput, compared to linear probing and Robin Hood, is due to the additional costs of maintaining displacement chains and possible rehashing.

**Limitations.** The following are some limitations of our benchmarks:

- The tests were performed using only 64-bit integer keys. String keys involve variable cost comparisons and different caching behavior.
- All tests used a single thread. Multiple threads in concurrent hash tables have different optimization strategies that cannot be captured through benchmarking.
- There is no SIMD support in any of the test implementations and thus cannot capture the current state of the art in hash table implementations, such as Swiss Table.
- The benchmarking uses a purely synthetic uniform random key distribution which may be inapplicable to skewed distributions seen in practice.
- Hardware and compiler optimization levels have a huge impact on absolute performance figures.

# 8 Conclusion and Future Work

The evolution of collision resolution techniques in hash tables from Luhn's work in 1953 to the latest cache-aware algorithms, together with an experimental comparison of four representative techniques, were studied in this research study. Below are our findings:

1. Open addressing outperforms separate chaining in throughput performance due mainly to its cache efficiency.
2. Robin Hood hashing yields the best performance/simplicity ratio and is therefore the best choice for general use.
3. Cuckoo hashing offers unmatched lookup speed despite its heavy space cost.

4. The trend towards SIMD-friendly data structure design based on metadata (i.e., Swiss Table and F14) is a new way of designing hash tables, where memory hierarchy is prioritized rather than counting operations.

Some promising areas for further investigation include:

- **Simplified Swiss Table.** Building a simple implementation of the SIMD metadata lookup of Swiss Table using C++ and SSE2 intrinsics will enable a comparison with our schemes.
- **String keys.** Running experiments on variable-length string keys will show how each scheme deals with the cost of the key comparison.
- **Concurrent variants.** Looking at lock-free or striped schemes will provide an important answer to an important question.
- **Real-world key distributions.** Running experiments with Zipfian or trace-driven workloads will show how each scheme performs on skewed accesses.

## References

[1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[2] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979.

[3] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1985, pp. 281–288.

[4] R. Pagh and F. F. Rodler, "Cuckoo hashing," in *Proc. 9th Annual European Symposium on Algorithms (ESA)*, Springer LNCS, vol. 2161, 2001, pp. 121–133.

[5] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Proc. 22nd International Symposium on Distributed Computing (DISC)*, Springer LNCS, vol. 5218, 2008, pp. 350–364.

[6] M. Kulukundis, "Designing a fast, efficient, cache-friendly hash table, step by step," CppCon 2017. [Online]. Available: https://www.youtube.com/watch?v=ncHmEUmJZf4

[7] Meta (Facebook), "F14 — Fast, Friendly, Folly Hash Map," Folly Library Documentation, 2019. [Online]. Available: https://github.com/facebook/folly/blob/main/folly/container/F14.md

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[9] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.

[10] M. Thorup, "Fast and powerful hashing using tabulation," *Communications of the ACM*, vol. 60, no. 7, pp. 94–101, 2017.

[11] S. Richter, V. Alvarez, and J. Dittrich, "A seven-dimensional analysis of hashing methods and its implications on query processing," *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 96–107, 2015.